Broadcasting semantics

Created On: Apr 27, 2017 | Last Updated On: Sep 29, 2025

Many PyTorch operations support NumPy's broadcasting semantics. See https://numpy.org/doc/stable/user/basics.broadcasting.html for details.

In short, if a PyTorch operation supports broadcast, then its Tensor arguments can be automatically expanded to be of equal sizes (without making copies of the data).

General semantics

Two tensors are "broadcastable" if the following rules hold:

- When iterating over the dimension sizes, starting at the trailing dimension, the dimension sizes must either be equal, one of them is 1, or one of them does not exist.

For Example:

```
>>> x=torch.empty(5,7,3)
>>> y=torch.empty(5,7,3)
# same shapes are always broadcastable (i.e. the above rules always hold)

>>> x=torch.empty((0,))
>>> y=torch.empty(2,2)
# x and y are not broadcastable, because the 0-sized dimension of x
# does not match the 2-sized dimension of y.

# can line up trailing dimensions
>>> x=torch.empty(5,3,4,1)
>>> y=torch.empty(  3,1,1)
# x and y are broadcastable.
# 1st trailing dimension: both have size 1
# 2nd trailing dimension: y has size 1
# 3rd trailing dimension: x size == y size
# 4th trailing dimension: y dimension doesn't exist

# but:
>>> x=torch.empty(5,2,4,1)
>>> y=torch.empty(  3,1,1)
# x and y are not broadcastable, because in the 3rd trailing dimension 2 != 3
```

If two tensors `x`, `y` are "broadcastable", the resulting tensor size is c s:

- If the number of dimensions of `x` and `y` are not equal, prepend 1 to the dimensions of the tensor with fewer dimensions to make them equal length.

Ask AI

- Then, for each dimension size, the resulting dimension size is the max of the sizes of `x` and `y` along that dimension.

For Example:

```
# can line up trailing dimensions to make reading easier
>>> x=torch.empty(5,1,4,1)
>>> y=torch.empty(  3,1,1)
>>> (x+y).size()
torch.Size([5, 3, 4, 1])

# but not necessary:
>>> x=torch.empty(1)
>>> y=torch.empty(3,1,7)
>>> (x+y).size()
torch.Size([3, 1, 7])

>>> x=torch.empty(5,2,4,1)
>>> y=torch.empty(3,1,1)
>>> (x+y).size()
RuntimeError: The size of tensor a (2) must match the size of tensor b (3) at nor
```

In-place semantics

One complication is that in-place operations do not allow the in-place tensor to change shape as a result of the broadcast.

For Example:

```
>>> x=torch.empty(5,3,4,1)
>>> y=torch.empty(3,1,1)
>>> (x.add_(y)).size()
torch.Size([5, 3, 4, 1])

# but:
>>> x=torch.empty(1,3,1)
>>> y=torch.empty(3,1,7)
>>> (x.add_(y)).size()
RuntimeError: The expanded size of the tensor (1) must match the existing size (7
```

Backwards compatibility

Ask AI

Prior versions of PyTorch allowed certain pointwise functions to execute on tensors with different shapes, as long as the number of elements in each tensor was equal. The

pointwise operation would then be carried out by viewing each tensor as 1-dimensional. PyTorch now supports broadcasting and the "1-dimensional" pointwise behavior is considered deprecated and will generate a Python warning in cases where tensors are not broadcastable, but have the same number of elements.

Note that the introduction of broadcasting can cause backwards incompatible changes in the case where two tensors do not have the same shape, but are broadcastable and have the same number of elements. For Example:

```
>>> torch.add(torch.ones(4,1), torch.randn(4))
```

would previously produce a Tensor with size: torch.Size([4,1]), but now produces a Tensor with size: torch.Size([4,4]). In order to help identify cases in your code where backwards incompatibilities introduced by broadcasting may exist, you may set *torch.utils.backcompat.broadcast_warning.enabled* to *True*, which will generate a python warning in such cases.

For Example:

```
>>> torch.utils.backcompat.broadcast_warning.enabled=True
>>> torch.add(torch.ones(4,1), torch.ones(4))
__main__:1: UserWarning: self and other do not have the same shape, but are broad
Changing behavior in a backwards incompatible manner to broadcasting rather than
```

Rate this Page ★ ★ ★ ★ ★

Send Feedback ›

‹ Previous
Autograd mechanics

Next
CPU threading and TorchScript inference ›

Built with the PyData Sphinx Theme 0.15.4.

Docs

Access comprehensive developer documentation for PyTo

View Docs ›

Ask AI

Tutorials

Get in-depth tutorials for beginners and advanced developers

View Tutorials >

Resources

Find development resources and get your questions answered

View Resources >

Stay in touch for updates, event info, and the latest news

First Name* Last Name* Email*

Select Country SUBMIT

By submitting this form, I consent to receive marketing emails from the LF and its projects regarding their events, training, research, developments, and related announcements. I understand that I can unsubscribe at any time using the links in the footers of the emails I receive. Privacy Policy.



Ask AI

© PyTorch. Copyright © The Linux Foundation®. All rights reser
Linux Foundation has registered trademarks and uses trademarks. For more information, including terms of use, privacy policy, and trademark usage, please see our Policies page. Trademark Usage. Privacy Policy.



Ask AI